Checking a coding assistant's advice

A 20-minute supervised activity in Session 21, week seven, for either track. Use 20 minutes of the existing independent-practice block; the rest remains project work. No new syntax, package, student account or paid subscription is required. Use only the practice function below, not the assessed project. Keep the learner's unassisted attempt.

0–5 minutes — Read and test without AI

Before showing any assistance, ask each learner what this function is supposed to calculate: price multiplied by quantity. Have them predict the result and then run it.

```
def total(price, quantity):
    return price + quantity

print(total(2000, 3))
```

Ask learners to explain the parameter names, return and the plus sign. The output is 2003; the required result is 6000. They have already practised functions and assert in Session 6. If they cannot explain these, return to the tiny Session 6 example before continuing.

5–10 minutes — Observe a limited request

The tutor may use a prepared local coding assistant on the demonstration computer. Do not ask students to sign up. Use this request:

"I wrote this function to calculate price multiplied by quantity. It gives 2003 for price 2000 and quantity 3. Explain what I should inspect and suggest two tests. Give a hint rather than rewriting the function."

Show the actual reply and label its source. If no assistant is available, use the following explicitly teacher-written response for critique; do not present it as real model output:

"Inspect the arithmetic operator. Try total(2000, 3) and total(1000, 0). Multiplication fits the stated calculation. Every bug in a total function is fixed by changing + to *."

Ask what is useful and what is unjustified. The final sentence is an overgeneralisation: the required calculation determines the correct operator; a different bug or a different function could require a different repair. An assistant may also ignore the request not to rewrite code. Discuss whether its response respected the task.

10–16 minutes — Check and explain

Learners make their own edit and test the function. Correct reference for the tutor:

```
def total(price, quantity):
    return price * quantity

assert total(2000, 3) == 6000
assert total(1000, 0) == 0
assert total(1000, 1) == 1000
```

Passing asserts print nothing. Tests support this small requirement; they do not prove correctness for every possible input. Ask each learner why zero quantity is useful to check. Do not add unfamiliar validation syntax or advanced features merely because an assistant suggests them.

16–20 minutes — Record and reflect

Save the original code, the advice, the learner's own edit and test results in AI_USE_LOG.md. Answer: what did you accept, what did you reject, and why? Explain the change without consulting the assistant. A learner can complete the entire activity by observing the tutor and critiquing the teacher-written example. No AI-generated solution is needed.

Close the assistant before returning to independent assessed work. The final project demonstration is always unaided. Read AI_USE_POLICY.pdf for the rules governing disclosure and supervision.